



EE 3501 Laboratory 4: GPIO, Scanning a Keypad

Name: _____

Date: _____

Learning Objective:

- Write a real-time polling routine to read inputs from a keypad.

Introduction

A Keypad as a Matrix of Switches

A keypad consists of matrix of switches used to connect an input pin to an output pin (Fig. 1). When a button is pressed, a switch is closed, and a connection is made between an input pin arranged in columns and an output pin arranged in rows.

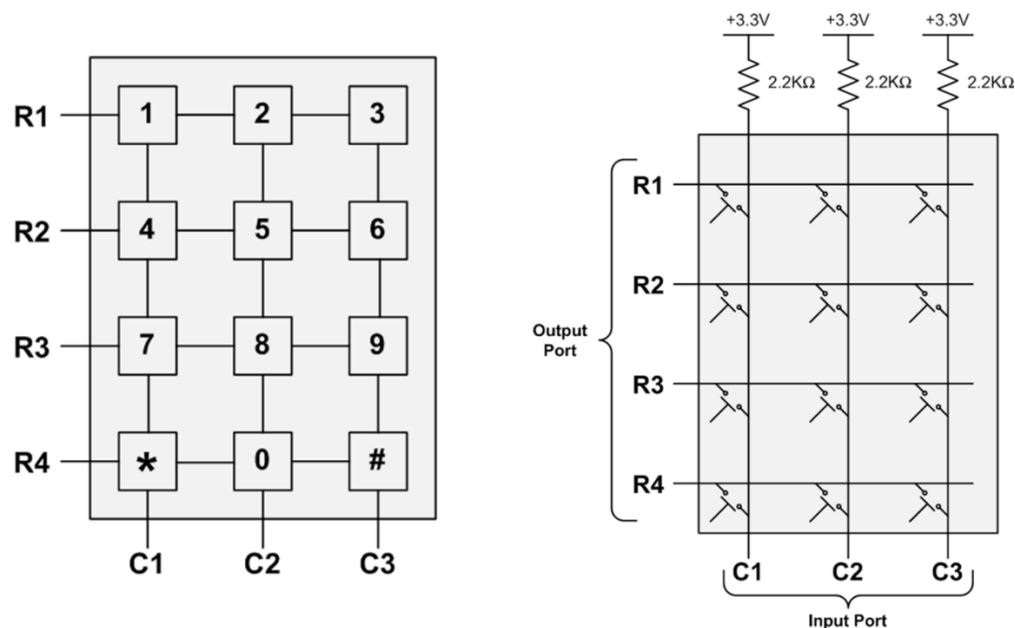


Figure 1 - A 4x3 Keypad with Underlying Switch Representation

The input pins of a keypad are typically connected to a high logic voltage via pull-up resistors. This means their default state is logic 1. The output pins can be set to either logic 1 or 0.

Mapping a Key Press to a Keypad Matrix Location

Consider a situation where all the output pins are set to 0 and no keys are pressed (Fig. 2). In this situation there is no connection between inputs and outputs and all input pins are logic 1.

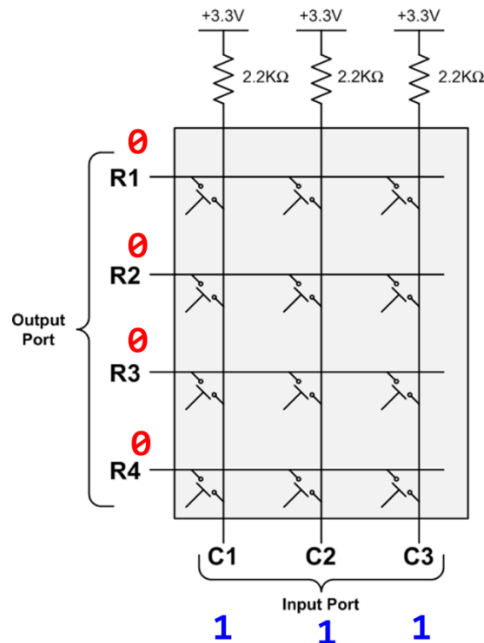


Figure 2 – Input Pin Values with Output Pins R1, R2, R3, R4 = 0000, and no Key Pressed

Now consider a situation where all the output pins are set to 0 and key “0” is pressed (Fig. 3 and Fig. 4). In this situation there is a connection between input pin C2 and output pin R4, which results in C2 being logic 0.

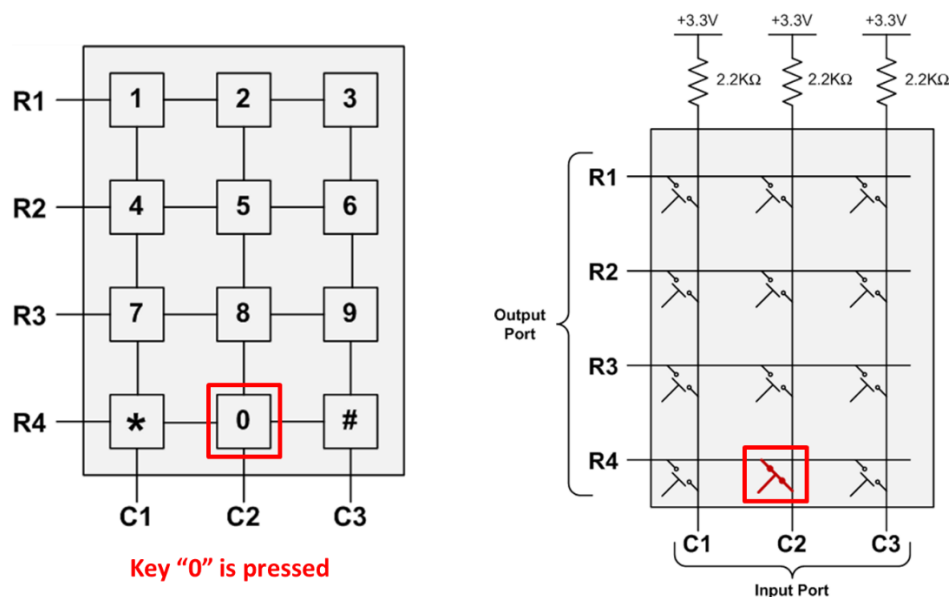


Figure 3 – Switch Closure Between R4 and C2 when Key “0” is Pressed

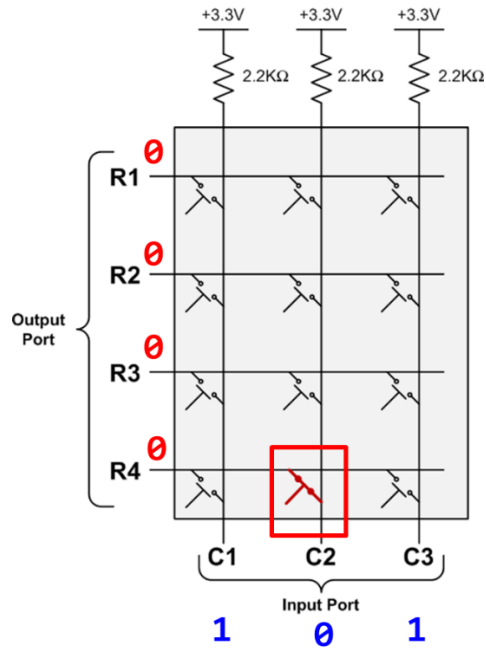


Figure 4 – Input Pin Values with Output Pins R1, R2, R3, R4 = 0000 and Key “0” Pressed

Fig. 4 illustrates that we can locate the input column of a key press when all the output rows are set to 0.

To obtain the row location as well as column location of a key press, we must successively set each output row to logic 0, while keeping the other output rows high and read the input pins each time. As shown in Fig. 5 – 8, the input pins will only indicate that a key in a column is pressed when the row in which the key is pressed is set to 0. This polling procedure allows us to determine the row and column location of a key press in the keypad matrix.

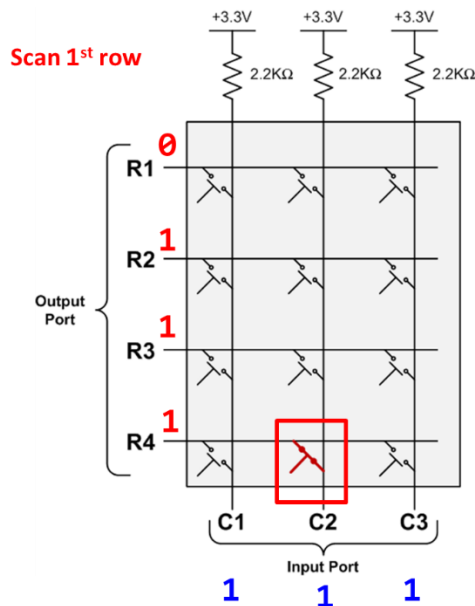


Figure 5 – Input Pin Values with Output Pins R1, R2, R3, R4 = 0111 and Key “0” Pressed

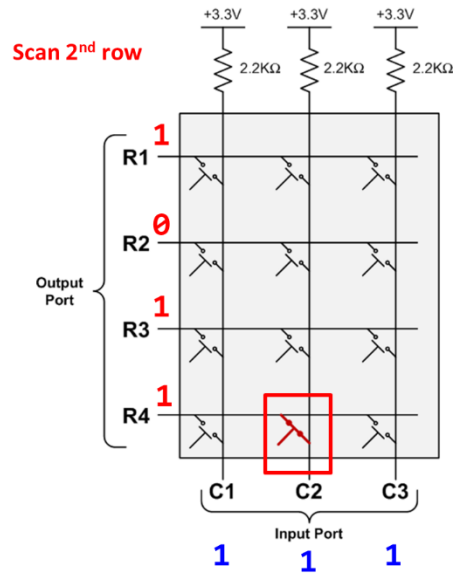


Figure 6 – Input Pin Values with Output Pins R1, R2, R3, R4 = 1011 and Key “0” Pressed

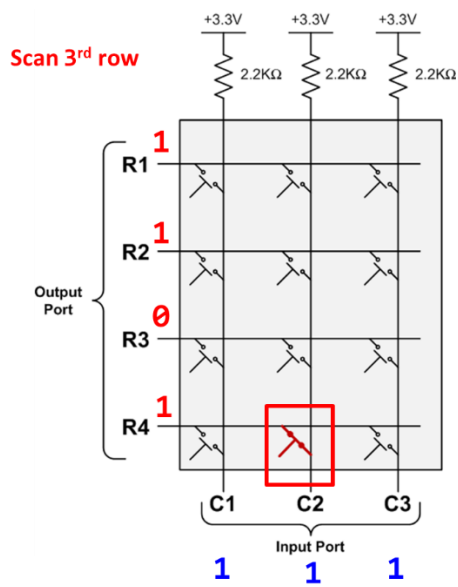


Figure 7 – Input Pin Values with Output Pins R1, R2, R3, R4 = 1101 and Key “0” Pressed

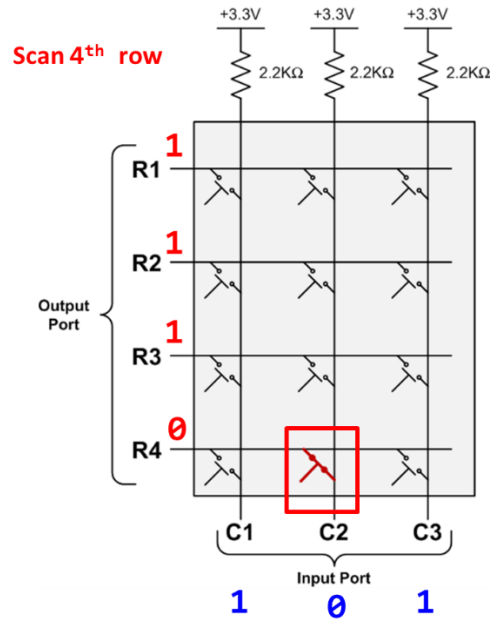


Figure 8 – Input Pin Values with Output Pins R1, R2, R3, R4 = 1110 and Key “0” Pressed

In this scenario, the keypad matrix location of key “0” is obtained through the successive “scanning” of rows. The location is row 4 and column 2.

Laboratory Procedure:

For this lab, you will create a C program that will use a polling procedure to read a key press from your keypad and output the corresponding character to the terminal. You will also modify this initial program to read a five-character sequence and turn LED1 on if the sequence matches a code.

Your keypad will look like Fig. 9. Your output row pins and input column pins are arranged as shown in the figure.

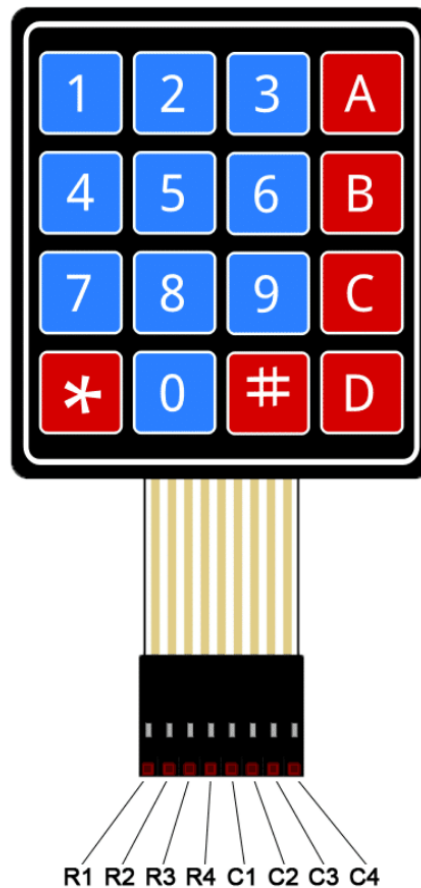


Figure 9 - Keypad Pinout Diagram

Activity 1: Polling Procedure to Read a Key Press

1. Connect the row pins and column pins of your keypad to GPIO pins PA6, PA7, PB6, PC7, PA9, PA8, PB10 and PB4 by connecting the female connector of your keypad to the morpho headers of your microcontroller board. Refer to [NUCLEO-F401RE | Mbed](#) for pinout details of your microcontroller board.
2. Create and name your project using the following format.
EE3501_Lab04a_lastname_firstinitial
3. Refer to the Semihosting Set-Up Instructions document to set up your project for semihosting.
4. Return to the Pinout & Configuration tab of STM32CubeMX and configure pins PA6, PA7, PB6 and PC7 for GPIO_Output and pins PA9, PA8, PB10 and PB4 for GPIO_Input. See Figure 10 below.

6. Under System Core, select GPIO and set COL1 to COL4 to Pull-Up. See Figure 12.

Pin ...	Signal o...	GPIO o...	GPIO m...	GPIO P...	Maximu...	User La...	Modified
PA6	n/a	Low	Output ...	No pull...	Low	ROW1	✓
PA7	n/a	Low	Output ...	No pull...	Low	ROW2	✓
PA8	n/a	n/a	Input mo...	Pull-up	n/a	COL2	✓
PA9	n/a	n/a	Input mo...	Pull-up	n/a	COL1	✓
PB4	n/a	n/a	Input mo...	Pull-up	n/a	COL4	✓
PB6	n/a	Low	Output ...	No pull...	Low	ROW3	✓
PB10	n/a	n/a	Input mo...	Pull-up	n/a	COL3	✓
PC7	n/a	Low	Output ...	No pull...	Low	ROW4	✓

PA9 Configuration :

GPIO mode

Input mode

GPIO Pull-up/Pull-down

Pull-up

User Label

COL1

Figure 12 - Pull-Up Option Selection for COL1 to COL4

7. Save your project.
8. Within main.c, write a polling procedure to output a logic 0 on the GPIO pins connected to ROW1, ROW2, ROW3, and ROW4 one at a time while keeping the other pins at logic 1. An output row will be held at logic 0 for 5 ms before the input column values are read and the output row will remain at logic 0 for 5 ms after the columns are read. Refer to the timing diagram of Fig. 13.

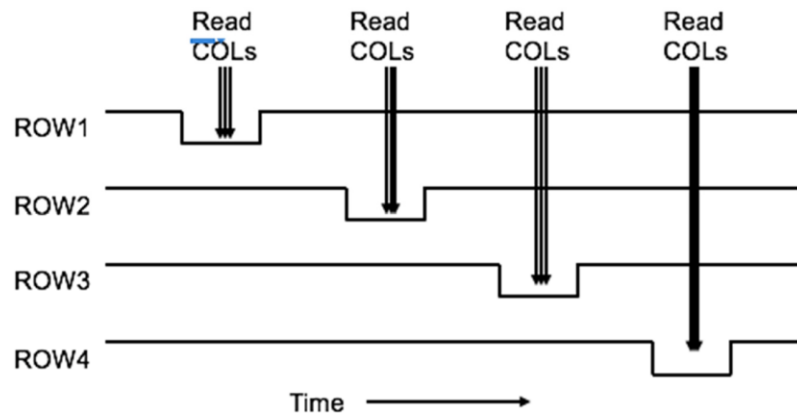


Figure 13 - Digital Output Timing Diagram for Keypad Rows

9. By using the polling procedure described in step 8, you can obtain the row and column location of a key press, which can be used to access elements of a char array that mimics the keypad layout. See an example char array structure below. Note that array index values start at 0, so `key_map[0][0]` is `'1'`, `key_map[1][2]` is `'6'`, and so on.


```
char key_map [4][4] = {
    {'1', '2', '3', 'A'}, //1st row
    {'4', '5', '6', 'B'}, //2nd row
    {'7', '8', '9', 'C'}, //3rd row
    {'*', '0', '#', 'D'}, //4th row
};
```

10. Use the polling procedure of step 8 and char array of step 9 to detect and locate a key press and then access the correct character from the char array. Output this character to the terminal using a `printf` statement.

I recommend that you use a structured hierarchical approach to create your program. One way to do this is create a function, that uses the polling procedure to return the character pressed by the keypad when called in the `main()` function. An example function declaration could be

```
char keypad_scan(void);
```

This function could also call a function that reads the state of the input columns to determine the column index of the key pressed. An example function declaration could be

```
int col_scan(void);
```

You can refer to the flowcharts in your Lab 4 Folder in D2L to help code these 2 functions

Activity 2: Read Five-Character Sequence and Turn LED1 on if Code Match

1. Modify the program you created in the previous procedure to read a five-character sequence. If the sequence matches the code 2753# then turn the green LED on. If not, the green LED remains off.
2. If the green LED is on, add to the program instructions that will turn it off when * is pressed.

You may find including the `<string.h>` library to be helpful. Specifically, you can compare two strings to determine if they are equal by using the `strcmp()` function.

Report Requirements

Lab Results

- Copy of the code used to implement the Polling Procedure to Read a Key Press program. (25 points)
- Video or instructor inspection of a user pressing ALL the keys on the keypad and the corresponding keys being displayed in the terminal screen. (25 points)
- Copy of the code used to implement Read Five-Character Sequence program. (25 points)
- Video or instructor inspection of a user pressing the correct Five-Character sequence and LED1 turning on in response, followed by the user pressing '*' and LED1 turning off in response. (25 points)

No formal lab report is needed for Lab 4.